

VITyarthi – Build Your Own Project

HOSTEL MANAGEMENT SYSTEM

PROJECT REPORT

Student	Himanshu lalit Fegade
Reg. No.	26BAI10713
Programme	B.Tech - Computer Science and Engineering (AI & ML)
Course	Python Essentials
Institution	VIT Bhopal University
Project Type	Modular Console-Based Python Application

Submitted as part of the VITyarthi flipped-course evaluation.

Table of Contents

1	Introduction
2	Problem Statement
3	Objectives
4	Functional Requirements
5	Non-Functional Requirements
6	System Architecture
7	Design Diagrams
8	Design Decisions & Rationale
9	Implementation Details
10	Screenshots / Results
11	Testing Approach
12	Challenges Faced
13	Learnings & Key Takeaways
14	Future Enhancements
15	References

1. Introduction

The Hostel Management System is a modular, menu-driven Python application developed to demonstrate how programming concepts can be applied to a practical hostel administration problem. It brings common hostel activities into one console application and separates each responsibility into an independent Python module.

The implementation uses standard Python data structures such as lists and dictionaries, functions, loops, conditional statements, searching, validation and modular imports. Demonstration data is loaded at startup so the complete workflow can be shown immediately.

Main functional areas:

- Student management
- Room allocation and room vacation
- Fee management
- Complaint registration and resolution
- Leave application and approval/rejection
- Dashboard and student reports

2. Problem Statement

Hostel administration involves maintaining student records, assigning rooms, recording fees, tracking complaints and processing leave applications. When these activities are handled separately, information can become difficult to organize. The project addresses this problem through a single modular console-based system.

3. Objectives

- Create a centralized application for common hostel operations.
- Apply modular programming using multiple Python files.
- Provide clear input/output workflows.
- Implement validation for common invalid inputs.
- Demonstrate CRUD-like student operations and status-based workflows.
- Provide dashboard and student-specific reports.
- Keep the application dependency-free and simple to run.

4. Functional Requirements

Module	Inputs	Outputs / Actions
Student Management	ID, name, course, branch, year, phone	Add, view, search, update, delete
Room Management	Student ID, room number	Allocate, view, vacate
Fee Management	Student ID, amount	Record and view fees
Complaint Management	Student ID, complaint	Register, view, resolve
Leave Management	Student ID, dates, reason	Apply, view, approve, reject
Reports	Dashboard or student ID	Summary and student report

The application follows a logical workflow: select a module, enter information, validate the input, perform the operation, display the result and return to the menu.

5. Non-Functional Requirements

Requirement	Project Interpretation
Performance	Operations should complete immediately for the intended small dataset.
Usability	Clear menu labels, prompts, confirmations and simple console interaction.
Reliability	Validation reduces duplicate IDs, invalid amounts and inconsistent room allocation.
Maintainability	Separate modules keep each feature focused.
Error Handling	Invalid IDs, full rooms, empty fields and invalid values produce messages instead of abrupt failure.
Scalability	Storage can later be replaced by SQLite/MySQL without redesigning every feature.
Resource Efficiency	Only standard Python functionality is required.

6. System Architecture

The application uses a simple modular architecture. The main controller presents the menu and routes the user to feature modules. Each feature module owns the functions for its domain. The current prototype uses in-memory Python lists and dictionaries as its storage layer.

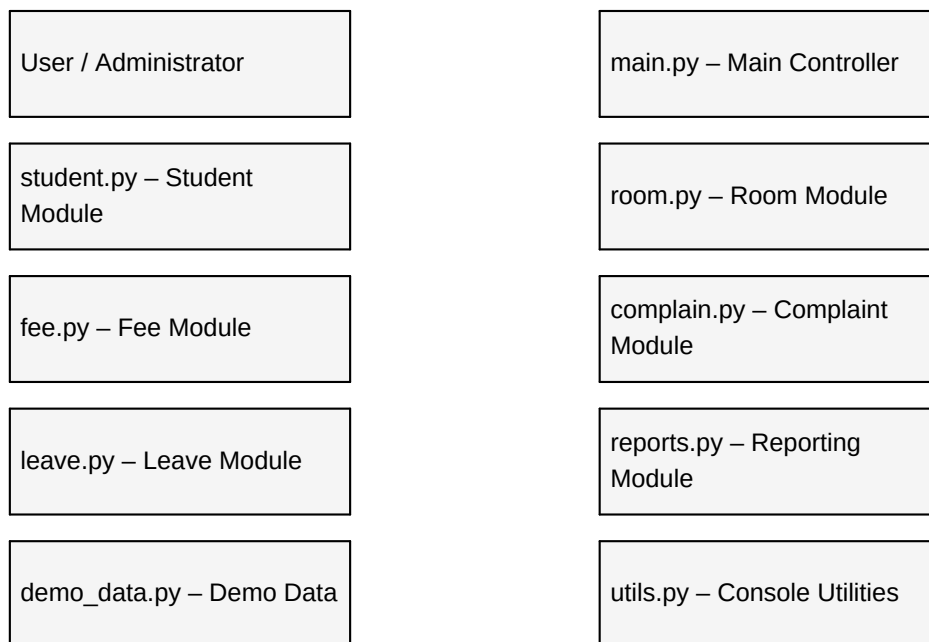


Figure 1. High-level component architecture

Data flow: User → main.py → selected module → validation/processing → in-memory data → result displayed to user.

7. Design Diagrams

7.1 Use Case Diagram

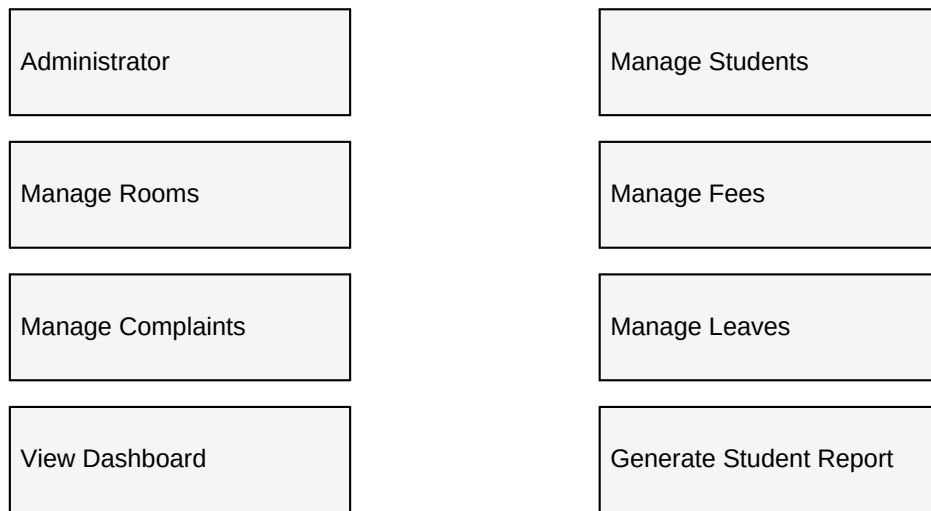


Figure 2. Use-case representation. The administrator interacts with all major system functions.

7.2 Workflow Diagram

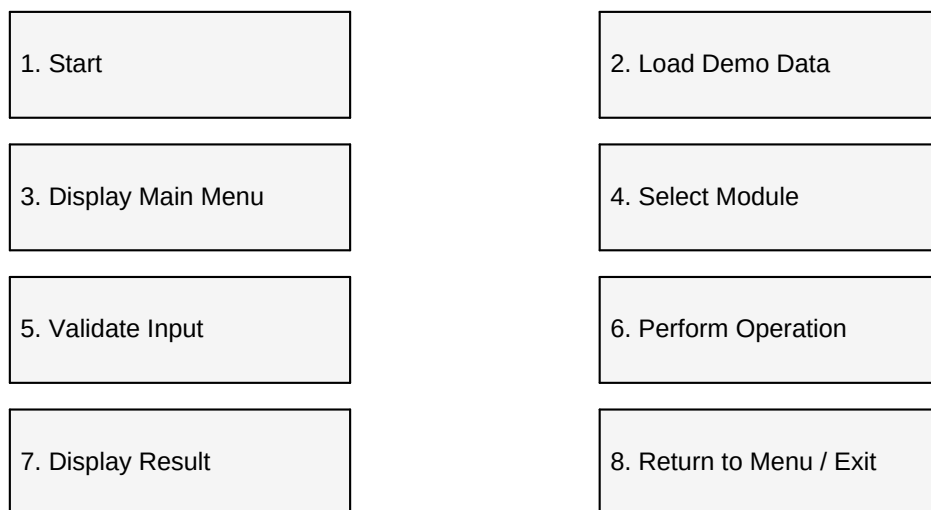


Figure 3. Application workflow

7.3 Sequence Diagram – Room Allocation

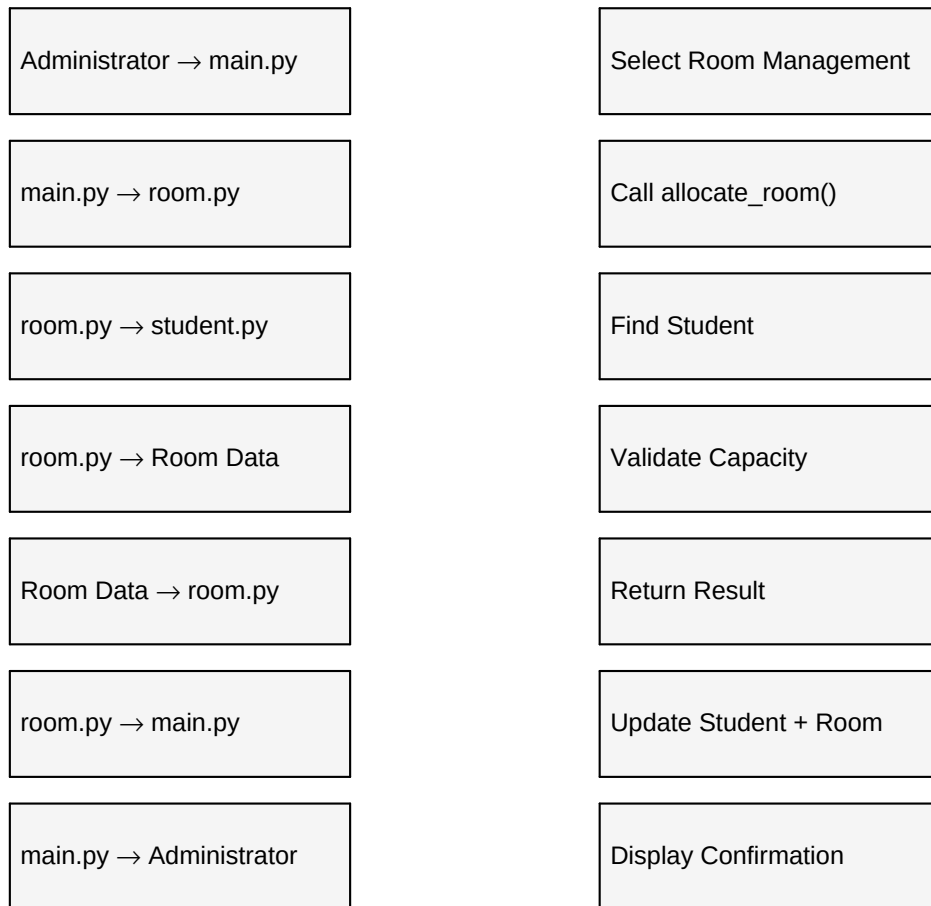


Figure 4. Sequence of a room-allocation operation

7.4 Class / Component View

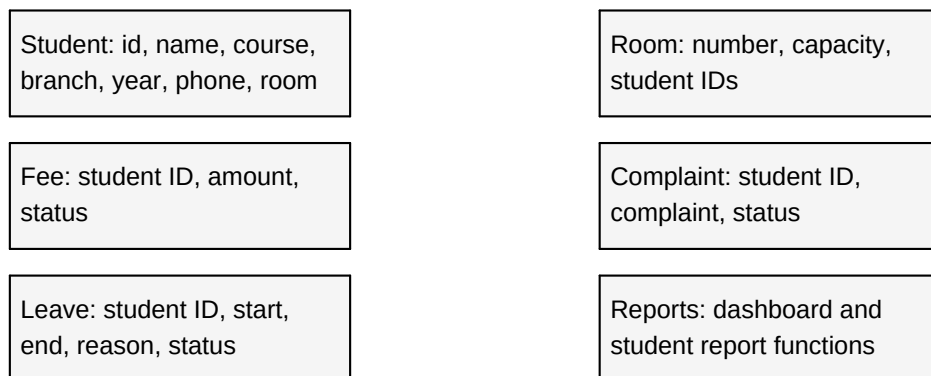


Figure 5. Core data entities and components

8. Design Decisions & Rationale

Modular Python structure

Major responsibilities are separated into individual files. This makes the implementation easier to read, maintain and demonstrate.

Console interface

A console interface keeps the project focused on Python fundamentals such as functions, modules, data structures, loops, conditionals and validation.

In-memory storage

Lists and dictionaries are used because they are directly relevant to introductory Python programming and keep the project easy to run without external services.

Demo data

Representative records are loaded at startup so the evaluator can immediately demonstrate the system.

9. Implementation Details

Student Management

Student records contain ID, name, course, branch, year, phone and room. The module supports adding, viewing, searching, updating and deleting records. Duplicate IDs are rejected.

Room Management

The prototype provides twelve rooms with a capacity of three beds each. Allocation checks student existence, current allocation, room validity and capacity. Vacating updates both room and student records.

Fee, Complaint and Leave Management

Fees store student ID, amount and status. Complaints begin as Pending and can be resolved. Leave applications store student, dates, reason and status and can be approved or rejected.

Reporting

The dashboard summarizes students, room capacity, occupancy, pending complaints, pending leaves and recorded fees. Student reports combine the student's room, fee, complaint and leave information.

10. Screenshots / Results

The final application is launched with ``python main.py``. It loads demonstration records and presents the main menu.

```
=====
HOSTEL MANAGEMENT SYSTEM
=====
1. Student Management
2. Room Management
3. Fee Management
4. Complaint Management
5. Leave Management
6. Dashboard & Reports
7. Exit
```

Area	Demonstration Result
Students	4 sample students
Rooms	A101 and A102 contain sample allocations

Area	Demonstration Result
Fees	2 recorded fee entries
Complaints	2 sample complaints with different statuses
Leaves	2 sample leave applications with different statuses

11. Testing Approach

Testing covers both normal operations and invalid-input cases. The final Python files were syntax-checked and the demo-data initialization was executed successfully after the room-numbering issue was corrected.

ID	Scenario	Expected Result
T01	Add unique student	Student added
T02	Duplicate student ID	Rejected
T03	Search valid ID	Student displayed
T04	Search invalid ID	Not found message
T05	Allocate available room	Allocation succeeds
T06	Allocate full room	Rejected
T07	Positive fee amount	Fee recorded
T08	Invalid fee amount	Validation message
T09	Register complaint	Pending complaint created
T10	Resolve complaint	Status becomes Resolved
T11	Apply leave	Pending leave created
T12	Approve/reject leave	Status changes
T13	Dashboard	Summary displayed

12. Challenges Faced

- Keeping student and room data consistent.
- Handling invalid input without terminating the application.
- Separating features into modules while keeping imports manageable.
- Providing meaningful demo data for immediate evaluation.
- Maintaining simplicity while covering multiple functional requirements.

13. Learnings & Key Takeaways

- Practical use of Python modules and imports.
- Working with lists and dictionaries to model real-world data.
- Using functions to separate responsibilities.
- Implementing searching, validation and CRUD-like operations.
- Designing menu-driven workflows.
- Considering maintainability and future database integration.

14. Future Enhancements

- SQLite/MySQL persistence.
- Role-based authentication.
- Web or GUI interface.
- CSV/PDF report export.
- Automated notifications.
- Attendance and hostel-service integration.
- Advanced occupancy and hostel analytics.

15. References

- Python 3.x language and standard-library concepts.
- VITyarthi – Build Your Own Project submission guidelines supplied for the course.
- Project source code, README.md, statement.md and internal design documentation.

End of Project Report